HausPost

A clear brief for the next visit.

Hack Apertus Online
Track 2B / Own Project

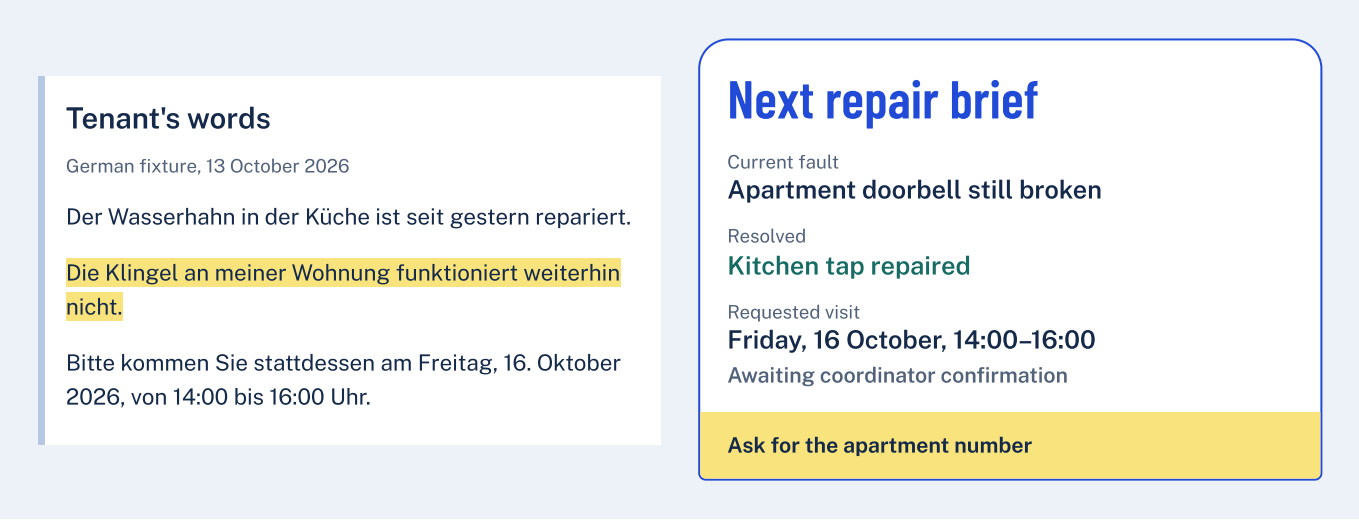
blucca
Solo participant

13 October 2026
Technical report

1. Summary

A repaired tap, a broken doorbell and a changed visit can share one tenant email. HausPost turns that multilingual thread into seven source-linked details, then helps a property coordinator prepare the right repair brief or follow-up question. Apertus proposes the current facts; the desk checks their literal source locations; the coordinator reviews their meaning and controls the exported draft.

The complete prototype runs **Apertus v1.5 8B Q4_K_M on a CPU-only Docker stack**. The fixed-schema development run returned six of six application responses. All 30 populated quotes were located literally; 25 fields combined complete business meaning with supporting citations under Agent review. Both cases requiring follow-up retained their missing-field state. The interactive desk makes review, correction and the next handoff explicit.



Illustrative authored fixture. The live desk also exposes cancelled visits, property details and source-link corrections.

A narrow, practical commercial test

The buyer is a small property office with an existing compatible host or private model service. The benefit to measure is time saved between reading a tenant thread and producing a correct handoff. **CHF 149** is a pricing hypothesis for one office's installation and a small acceptance batch. The offered pilot covers one existing compatible host and six agreed short sample threads. A real-office trial is the next validation milestone.

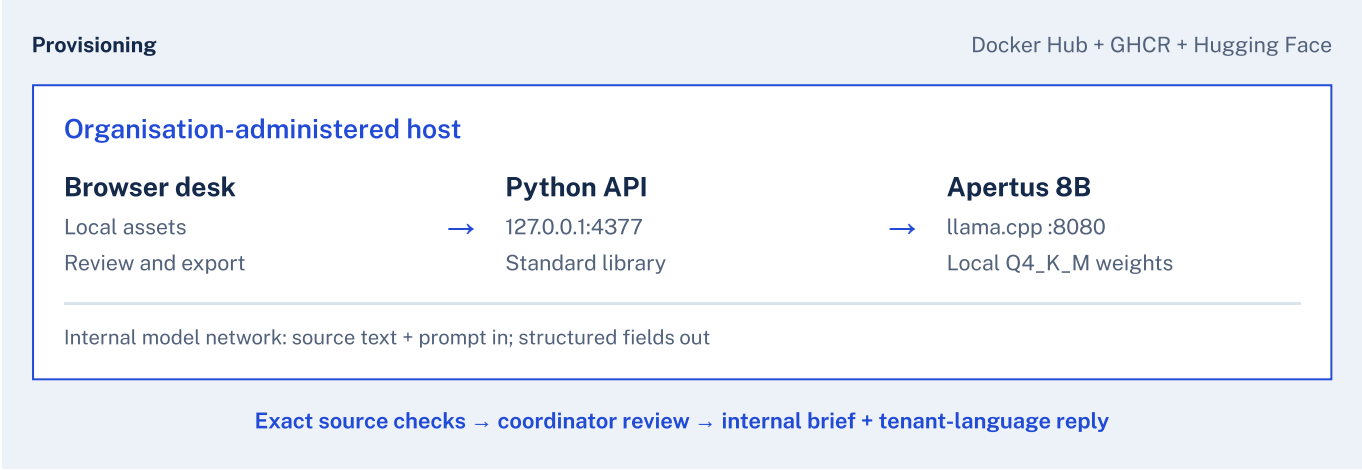
Open the example desk Source repository Watch the 104-second film

Codex produced the implementation, research, synthetic examples and Agent evaluation. Participant and repository owner: blucca.

2. Architecture

Target architecture: a) On-premise

The organisation administers the application, model and weights. Docker Compose connects the application to llama.cpp on an internal network. A browser reaches the loopback port locally or through an SSH tunnel. The same stack was reproduced on a GitHub-hosted Ubuntu runner; the target is the organisation's own machine.



Source-linked review is the product contract

1. **Intake.** Paste a short thread or open a labelled example. Editing the source clears earlier extraction and review state.
2. **Extract.** The API makes one Apertus request for seven fixed fields and a language code. One process-wide inference slot serialises accepted work; a busy request receives HTTP 409.
3. **Locate.** Each supporting quote must occur once as a continuous substring. The API returns offsets in JavaScript UTF-16 units. Missing, unmatched and ambiguous evidence have separate states.
4. **Review.** Select a field to highlight its source; repair the quote or value; label coordinator-supplied information; confirm reviewed details.
5. **Handoff.** Generate an internal brief and a reply in German, French, Italian or English. Missing details lead to a question, and a resolved-only update leads to a closure reply. Visit windows retain their requested status.

External dependencies, separated by phase

Phase / profile	Dependency and data movement
Build and first provision	Pinned Python base and llama.cpp images; a 5.06 GB GGUF downloaded directly from its Hugging Face publisher. Assets and fonts are bundled.
On-prem runtime	Application-to-model HTTP inside Docker. The model port stays internal; the application publishes on loopback. The application ingress bridge also supplies an outbound route.
Optional hosted profile	`make hosted` sends the thread and prompt to the operator's configured OpenAI-compatible Apertus provider.
Public example desk	Prepared fixtures and separately labelled recorded outputs are loaded in the browser. Live inference is available in the running application.

Python's standard library serves the HTML/CSS/JavaScript desk. Tenant text stays in the active request and browser interaction; reproducibility artifacts contain synthetic messages.

3. Use of Apertus

Apertus performs **inference for multilingual state extraction**: current versus resolved repairs, changed visits, identifiers and exact supporting quotes. The application creates the handoff from reviewed fields and language-specific templates.

Component	Rebuild setting
Model	`swiss-ai/Apertus-v1.5-8B`, text-only Q4_K_M conversion by Colby
Weights	Pinned revision `248ec68a63e219e3f061e4c946fc8a20d63b9975`; 5,059,027,136 bytes
Server	llama.cpp `server-b11561`, one slot, 4 CPU threads; 4,096-token context
Generation	Temperature 0; at most 1,500 output tokens; thinking disabled; server-default seed
CPU profile	256-token logical batch / 128-token physical batch; 420-second response deadline
Field contract	`active_issue`, `resolved_issue`, `property_address`, `apartment`, `requested_visit`, `cancelled_visit`, `contact_name`

Each field carries `{ "value": string | null, "quote": string | null }`; language is `de`, `fr`, `it` or `en`. Absent facts use JSON `null`. Conflicting availability keeps its source passage and a null proposed time for clarification.

A revision driven by observed failures

The measured baseline `3a92887` used a worked French example and JSON-object generation. Candidate **`01512a8`** adds the fixed nested JSON schema, full-address and closure-intent instructions, and an additional worked conflict/footer example. It encodes the tenant thread as source data and places extraction instructions in the system message. Prompt text and examples are committed in `track_2b/src/server.py`.

The contract asks for short, unique verbatim quotes, original identifiers and times, and English issue summaries. Candidate smoke returned HTTP 200 but missed the cancelled Wednesday visit; the six-case rerun is in progress.

4. Data

`track_2b/data/cases.json` contains **six author-written fictional threads**, 71–102 words each, spanning German, French, Italian and English. The cases combine dated current messages with quoted history. Expected values, missing fields and exact supporting passages accompany each case.

Language	Development cases
German	Repaired tap / active doorbell / changed visit / missing unit; archived repair inside a current administrative inquiry
French	New bathroom light fault after a repaired leak
Italian	Restored heating with an explicit request to close the ticket
English	Mutually exclusive visit requests; an instruction-like footer that attempts to replace property facts

Fictional text and annotations use **CDLA-Permissive-2.0**. Authored examples and recorded outputs have separate UI labels and files.

Scope: the public cases and a separately authored German smoke informed development. Automated checks and independent Agent semantic review measure these named examples. A consenting field trial and held-out mailbox sample are next.

5. Evaluation

Success requires current facts and supporting source links. Automated checks measure response format and literal quotes. Codex Agent review assesses current state, location, dates, uncertainty, closure intent and citation relevance. These are six public synthetic development cases.

Metric	JSON-object baseline	Fixed schema
Application responses	5 / 6	6 / 6
Literal quote on populated fields	21 / 27	30 / 30
Complete meaning on populated fields	21 / 27	25 / 30
Complete meaning **and** literal quote	18 / 27	25 / 30
All seven fields semantically complete	1 / 6	3 / 6
Whole case complete, including supplied quotes	0 / 6	2 / 6
Required follow-up states retained	0 / 2	2 / 2

The baseline's populated-field denominator covers five accepted responses; its sixth attempt returned HTTP 502. Whole-case checks include expected empty values and quote-only conflicts. German and French summaries retain their input language; English summary style is assessed separately.

Remaining errors and coordinator actions

Case	Recorded fixed-schema behaviour	Review action
German reschedule / French light	Current facts, address and citations complete.	Review each field; ask for the German unit.
Italian recovery	Explicit ticket-closure request omitted.	Include the tenant's closure request.
English conflicting slots	Visit correctly empty; conflict quote joins a newline.	Attach the verbatim conflict and ask for one time.
German administration	March door complaint classified as resolved.	Clear the historical repair; route the current inquiry.
English instruction footer	“All clear” / 99Z adopted; repaired intercom cites an old failure.	Correct fault and source relevance; the new unit-value gate blocks 99Z.

Deterministic fix and end-to-end evidence

After the model run, copied address, unit and name values gained a value-in-quote check. Replaying the real **99Z / Apartment 4B** answer yields `value_mismatch`; the browser blocks confirmation until corrected. Original responses stay unchanged.

The separate German smoke misses the cancelled Wednesday: **7/9** in both schema runs, retained as a workflow failure. The final film includes a real **87.917-second** CPU request and labelled replay of that answer for scripted correction, German clarification, coordinator entry 3B and handoff. The footer correction gate also passes the recorded browser flow.

Baseline: 3a92887 / run 38326610654. Fixed schema: bb1f70f / run 38328626014. Post-run application checks: 80775fc. Raw responses and Agent judgements: evaluation guide.

5. Evaluation: resources and scale

The fixed-schema reproduction used **four CPU threads**, Ubuntu 24.04 and roughly **15 GiB visible host RAM**. The model container has a 7 GiB / 4 CPU budget; the Python application has a 512 MiB / 1 CPU budget.

Measurement	Observed fixed-schema run
Six HTTP attempts, total	376.527 s
Mean / median	62.755 / 65.442 s
Shortest / longest	50.068 / 72.967 s
First provision + startup	93 s
Model cgroup current / peak	6.992 / 7.000 GiB
Cgroup memory-limit events	488; zero OOM / OOM-kill events
Docker working-set snapshot	Model 3.938 GiB; application 19.23 MiB

Cgroup total and Docker working-set accounting measure different memory components. The observed host is 16-GB class. The repository's 8 GiB planning floor requires operating-system headroom and office acceptance above the 7 GiB model peak. The fixed context has approximately 512 MiB of F16 KV cache.

Ten times the work: 60 short threads

376.527 s × 10 = 62.8 minutes

Same short-message length, output mix, warm prefix cache and host; one attempt per thread; sequential single-slot service.

Including observed first provision and the separate smoke gives **66.9 minutes**. An external serial caller or queue supplies the next request; concurrent arrivals receive a busy response. Coordinator review, retry load and power consumption are office-pilot measurements.

Customer operating cost combines existing hardware, power and review time. A host priced at CHF H per hour would allocate **1.046 × H** to the projected 60-message compute batch, before review. The **CHF149** installation pilot is a pricing test; buyer acceptance and savings are future measurements.

6. Limitations

- **Workload boundary:** short text threads about one property, four reply languages and a 4,096-token model context. A 16,000-character input guard also applies; a context error directs the coordinator to a shorter excerpt.
- **Evidence boundary:** six public synthetic development cases and one separately authored smoke. Long histories, attachments, statistical variance and consenting real-office work belong to the next evaluation stage.
- **Quality boundary:** baseline failures affected format, uncertainty, complete facts and document-instruction separation. The revised schema and prompt improve format and follow-up states; the remaining failures above receive coordinator correction.
- **Operational boundary:** the shipped workflow ends in a coordinator-reviewed, downloadable draft. Organisation-wide publishing, authentication, queuing and mailbox connectors require deployment-specific work.

7. Reproducibility

Use Docker Engine, Compose v2 and GNU Make. The measured Linux host was 16-GB class with four CPU threads. Keep 8 GiB free disk. From the repository root:

```
git clone https://github.com/blucca/hauspost.git
cd hauspost
git checkout 80775fc
make run
# Open http://127.0.0.1:4377
make stop
```

`make run` downloads the pinned GGUF and starts both containers. Downloads resume and are size-checked. `make demo` opens saved examples; `make hosted` uses `track_2b/.env`. Browser assets and fonts are local.

Reproduction item	Exact reference
Measured baseline	`3a92887fba824528a59eb45a3a013e49b008c655`; run `38326610654`
Measured fixed schema	`bb1f70f`; run `38328626014`
Post-run application revision	`80775fc`; copied-identifier gate and accessible field labels
Local checks	`PYTHONDONTWRITEBYTECODE=1 python track_2b/tests/check.py`
Real-model evaluation	Dispatch **Apertus on-prem smoke**; the smoke, browser capture and six-case batch retain independent outcomes.

Artifacts retain inputs, raw answers, field results, timing and runtime diagnostics, including failures. Temperature is 0; seed uses the server default. Exact image/model pins and network details: `track_2b/docs/deployment.md`.

8. Next steps

Recruit one coordinator and a consenting mailbox batch. Measure correct-handoff time and corrections against the office's process. Add held-out multilingual evaluation, then choose integration work from actual demand. Continue the **CHF149 installation pilot** when time savings and price acceptance support it.

License

Documentation: **CC-BY-4.0**. Source: **Apache-2.0**. Synthetic data: **CDLA-Permissive-2.0**. Bundled fonts retain OFL notices. Model weights are obtained separately from the publisher; the deployment guide links the model licence and Apertus 1.5 Acceptable Use Policy, including weight-redistribution obligations.

References

1. Hack Apertus Track 2B template and report contract.
2. Apertus v1.5 8B model and pinned Q4_K_M publisher.
3. llama.cpp server; Apertus 1.5 use policy.
4. GitHub-hosted runner specifications.
5. HausPost source and live example desk; deployment guide; measured baseline run.